HDOC DAY – 71

Algorithm

1. Start.
2. Read the number of rows and columns of matrices A and B.
3. Dynamically allocate memory for both matrices using malloc().
4. Check whether memory allocation was successful.
5. Read matrix elements using pointer notation.
6. For addition, check whether both matrices have the same dimensions. Add corresponding elements using pointers.
7. For subtraction, check whether both matrices have the same dimensions. Subtract corresponding elements using pointers.
8. For multiplication, check whether the number of columns of A equals the number of rows of B.
9. Multiply the matrices using three nested loops and pointer notation.
10. Display the results.
11. Release dynamically allocated memory using free().
12. Stop.

C PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>


void readMatrix(int *a, int r, int c)

{

  int i, j;


  for (i = 0; i < r; i++)

  {

    for (j = 0; j < c; j++)

    {
```

```
            scanf("%d", (a + i * c + j));
        }
    }
}

void displayMatrix(int *a, int r, int c)
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d ", *(a + i * c + j));
        }
        printf("\n");
    }
}

void addMatrix(int *a, int *b, int *result, int r, int c)
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            *(result + i * c + j) =
```

```
            *(a + i * c + j) + *(b + i * c + j);
      }
   }
}

void subtractMatrix(int *a, int *b, int *result, int r, int c)
{
   int i, j;

   for (i = 0; i < r; i++)
   {
      for (j = 0; j < c; j++)
      {
         *(result + i * c + j) =
            *(a + i * c + j) - *(b + i * c + j);
      }
   }
}

void multiplyMatrix(int *a, int *b, int *result,
                int r1, int c1, int c2)
{
   int i, j, k;

   for (i = 0; i < r1; i++)
   {
      for (j = 0; j < c2; j++)
      {
```

```
            *(result + i * c2 + j) = 0;

            for (k = 0; k < c1; k++)
            {
               *(result + i * c2 + j) +=
                  *(a + i * c1 + k) *
                  *(b + k * c2 + j);
            }
         }
      }
}

int main()
{
   int r1, c1, r2, c2;
   int *A, *B, *result;

   printf("Enter rows and columns of Matrix A: ");
   scanf("%d %d", &r1, &c1);

   printf("Enter rows and columns of Matrix B: ");
   scanf("%d %d", &r2, &c2);

   A = (int *)malloc(r1 * c1 * sizeof(int));
   B = (int *)malloc(r2 * c2 * sizeof(int));

   if (A == NULL || B == NULL)
   {
```

```
    printf("Memory allocation failed.\n");
    free(A);
    free(B);
    return 1;
}

printf("Enter elements of Matrix A:\n");
readMatrix(A, r1, c1);

printf("Enter elements of Matrix B:\n");
readMatrix(B, r2, c2);

/* Addition and Subtraction */
if (r1 == r2 && c1 == c2)
{
    result = (int *)malloc(r1 * c1 * sizeof(int));

    if (result == NULL)
    {
        printf("Memory allocation failed.\n");
        free(A);
        free(B);
        return 1;
    }

    addMatrix(A, B, result, r1, c1);

    printf("\nAddition:\n");
```

```
    displayMatrix(result, r1, c1);

    subtractMatrix(A, B, result, r1, c1);

    printf("\nSubtraction:\n");
    displayMatrix(result, r1, c1);

    free(result);
}
else
{
    printf("\nAddition and subtraction are not possible.\n");
}

/* Multiplication */
if (c1 == r2)
{
    result = (int *)malloc(r1 * c2 * sizeof(int));

    if (result == NULL)
    {
        printf("Memory allocation failed.\n");
        free(A);
        free(B);
        return 1;
    }

    multiplyMatrix(A, B, result, r1, c1, c2);
```

```
        printf("\nMultiplication:\n");
        displayMatrix(result, r1, c2);

        free(result);
    }
    else
    {
        printf("\nMultiplication is not possible.\n");
    }

    free(A);
    free(B);

    return 0;
}
```

TEST CASE TABLE:

Test Case	Matrix A	Matrix B	Addition	Subtraction	Multiplication
1	1 2; 3 4	5 6; 7 8	6 8; 10 12	-4 -4; -4 -4	19 22; 43 50
2	2 3; 4 5	1 2; 3 4	3 5; 7 9	1 1; 1 1	11 16; 19 28
3	2×3 matrix	3×2 matrix	Not possible	Not possible	Possible
4	2×2 matrix	3×3 matrix	Not possible	Not possible	Not possible